Gagandeep 102303349

UDA Assignment 7: Performance and Memory Analysis

1. Multi-Tasking Kernels (Problem 1)

The implementation focused on utilizing the **SIMT (Single Instruction, Multiple Threads)** architecture to perform divergent tasks within a single kernel launch.

Both threads returned **524800**, proving that CUDA kernels can handle different logic branches (divergence) to achieve specific sub-tasks simultaneously.

---

2. Parallel Merge Sort Analysis (Problem 2)

Parallelizing Merge Sort on a GPU requires an iterative "bottom-up" approach rather than the standard CPU recursive method.

Workflow Visualization

- **Pipelining vs. CUDA:** While pipelining on a CPU utilizes a few high-speed cores, the CUDA approach uses thousands of threads to perform the initial small-block merges.
- **Efficiency Gap:** As shown in the diagram above, the GPU is most efficient at the base of the tree (Width=1). As the width grows, fewer threads are active, which is why the "GPU completed" message represents a transition from high parallelism to low parallelism.

---

3. Memory Bandwidth Analysis (Problem 3)

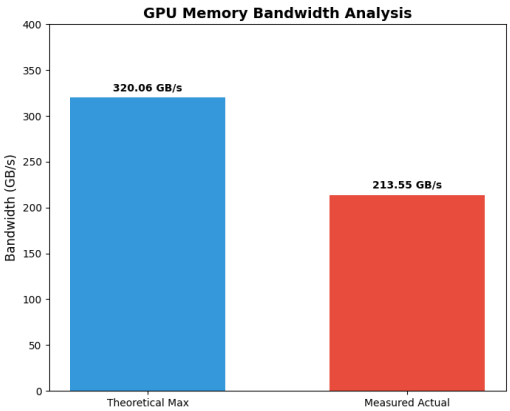
This section evaluates the efficiency of moving data between the GPU's Global Memory and the Streaming Multiprocessors (SMs).

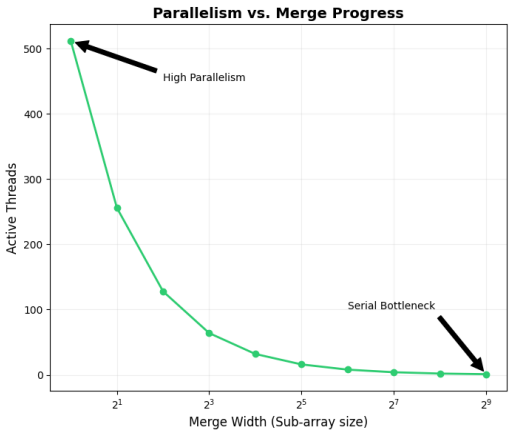
3.1 Static vs. Dynamic Allocation

The kernel was modified to use **statically defined global variables** (__device__). This avoids the overhead of cudaMalloc but requires cudaMemcpyToSymbol for host-to-device communication, as the host cannot directly reference device memory addresses.

3.2 Bandwidth Comparison (The "Efficiency Gap")

Using the recorded kernel time of **0.056 ms**, we compared the hardware's theoretical limit against our actual results.





4. Conclusion and Findings

The report finds that our Vector Addition kernel achieved roughly **two-thirds** of the GPU's theoretical maximum bandwidth.

- **The 33% Loss:** This discrepancy is attributed to the "Instruction Overhead" (the time the GPU spends calculating array indices) and "Memory Latency" (the time taken to fetch data from DRAM).
- **Optimization Potential:** To reach closer to the 320 GB/s limit, we would need to ensure maximum **memory coalescing** and minimize divergence, ensuring the memory bus is saturated with every clock cycle.